APPENDIX FOR EDGE BOARD:

```
module
mot( input clk,
input rst_n,
input echo1,
input echo2,
// Front ultrasonic echo
// Back ultrasonic echo
output reg trig1, // Front ultrasonic trigger
output reg trig2, // Back ultrasonic trigger
output reg pwm_servo, // PWM to servo motor
output reg [5:0] motor, // Motor control outputs [IN1, IN2, ENA, IN3, IN4, ENB]
output reg LED1, // Obstacle front
output reg LED2, // Motor control outputs [IN1, IN2, ENA, IN3, IN4, ENB]
output reg Led1, // Obstacle front
output reg Led2 // Obstacle back
);
// ----- Parameters -----
parameter CLOCK_FREQ = 50000000;
parameter PWM_FREQ = 50;
parameter PWM_PERIOD = CLOCK_FREQ / PWM_FREQ;
parameter MIN_PULSE = CLOCK_FREQ / 1000; // 1ms (0°)
parameter MAX_PULSE = CLOCK_FREQ / 500; // 2ms (180°)
parameter DIST_THRESH = 40;
parameter TRIG_PULSE_WIDTH = 500;
// ----- Servo rotation FSM -----
reg [1:0] servo_state = 0;
reg [31:0] servo_counter = 0;
reg [7:0] angle = 90;
```

```
reg [31:0] pulse_width;
always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
servo_state <= 0;
servo_counter <= 0;
angle <= 90;
end else begin
if (servo_counter >= 25_000_000) begin
servo_counter <= 0;
servo_state <= servo_state + 1;
case (servo_state)
2'd0: angle <= 135; // Right
2'd1: angle <= 90; // Center
2'd2: angle <= 45; // Left
2'd3: angle <= 90; // Back to Center
endcase

end else
servo_counter <= servo_counter + 1;
end
end
// ----- PWM generation for Servo -----
always @(*) begin
pulse_width = MIN_PULSE + ((MAX_PULSE - MIN_PULSE) * angle) / 180;
end
reg [31:0] pwm_counter = 0;
always @(posedge clk) begin
if (pwm_counter < pulse_width)
pwm_servo <= 1;
```

```
else
pwm_servo <= 0;
if (pwm_counter >= PWM_PERIOD)
pwm_counter <= 0;
else
pwm_counter <= pwm_counter + 1;
end
// ----- Ultrasonic Trigger Logic -----
reg [19:0] trig_timer1 = 0;
reg [19:0] trig_timer2 = 0;
always @(posedge clk) begin
trig_timer1 <= (trig_timer1 >= 1000000) ? 0 : trig_timer1 + 1;
trig1 <= (trig_timer1 < TRIG_PULSE_WIDTH) ? 1 : 0;
trig_timer2 <= (trig_timer2 >= 1000000) ? 0 : trig_timer2 + 1;
trig2 <= (trig_timer2 < TRIG_PULSE_WIDTH) ? 1 : 0;
end
// ----- Echo Measurement -----
reg [31:0] counter_echo1 = 0, counter_echo2 = 0;
reg [31:0] distance1 = 0, distance2 = 0;
reg prev_echo1 = 0, prev_echo2 = 0;
always @(posedge clk) begin
// Front echo
prev_echo1 <= echo1;
if (echo1 && !prev_echo1)
counter_echo1 <= 0;
else if (echo1)
counter_echo1 <= counter_echo1 + 1;
else if (!echo1 && prev_echo1)
distance1 <= counter_echo1 / 2900;
```

```
// Back echo
prev_echo2 <= echo2;
if (echo2 && !prev_echo2)
counter_echo2 <= 0;
else if (echo2)
counter_echo2 <= counter_echo2 + 1;
else if (!echo2 && prev_echo2)
distance2 <= counter_echo2 / 2900;
end
// ----- LED indication -----
reg [23:0] led1_timer = 0, led2_timer = 0;
always @(posedge clk) begin
if (distance1 > 0 && distance1 < DIST_THRESH)
led1_timer <= 24'd5000000;
else if (led1_timer > 0)
led1_timer <= led1_timer - 1;
LED1 <= (led1_timer > 0);
Led1 <= (led1_timer > 0);
if (distance2 > 0 && distance2 < DIST_THRESH)
led2_timer <= 24'd5000000;
else if (led2_timer > 0)
led2_timer <= led2_timer - 1;
LED2 <= (led2_timer > 0);
Led2 <= (led2_timer > 0);
end
// ----- Motor Control -----
reg [31:0] pwm_counter_motor = 0;
reg pwm_motor;
```

```
wire [31:0] full_speed = 32'd750000; // Full speed PWM
wire [31:0] slow_speed = 32'd250000; // Slow speed when obstacle detected
reg [31:0] selected_speed;
always @(*) begin
if (distance1 < DIST_THRESH && distance1 > 0)
selected_speed = slow_speed;
else
selected_speed = full_speed;
end
always @(posedge clk) begin
pwm_motor <= (pwm_counter_motor < selected_speed) ? 1 : 0;
pwm_counter_motor <= (pwm_counter_motor >= 1000000) ? 0 : pwm_counter_motor + 1;
end
// Direction Control
always @(*) begin

if (distance1 < DIST_THRESH && distance1 > 0) begin
// Turn right when obstacle in front
motor[0] = 1; // IN1
motor[1] = 0; //
IN2 motor[3] = 1; //
IN3 motor[4] = 0; //
IN4
end
motor[2] = pwm_motor; // ENA
motor[5] = pwm_motor; // ENB
end
endmodule
```